

Latent Structural Property Profiling in Code Large Language Model

Jyoti Prakash Meher¹, *Member, IEEE*, Rajib Mall², *Senior Member, IEEE*

¹Department of Computational and Data Science, National Institute of Technology, Warangal, India, 506004

²Department of Computer Science and Engineering, Shiv Nadar Institute of Eminence, New Delhi, India

The abstract goes here.

Index Terms—IEEE, IEEEtran, IEEE Transactions on Magnetics, journal, LATEX, magnetics, paper, template.

I. INTRODUCTION

THIS demo file is intended to serve as a “starter file” for IEEE TRANSACTIONS ON MAGNETICS journal papers produced under LATEX using IEEEtran.cls version 1.8b and later.

II. PROPOSED APPROACH

We characterize the internal representation of program structure in CodeLLMs along three complementary dimensions: syntax, control flow, and data flow. These dimensions are selected because they represent progressively richer forms of program organization. Syntax captures the structural arrangement of program elements; control flow captures the possible execution transitions among program units; and data flow captures the propagation of values through definition-use relationships. The objective is not to determine whether these properties can be decoded from a model’s hidden states using an auxiliary classifier. Instead, we measure whether the model’s representations exhibit selective geometric organization with respect to these properties under controlled structural variation.

Let M denote a CodeLLM with L transformer layers. For a program x , tokenized as $x = (x_1, \dots, x_T)$, the hidden representation produced by layer l is

$$H^{(l)}(x) \in \mathbb{R}^{T \times d},$$

where d is the hidden-state dimensionality. For each structural dimension q , where $q \in \{\text{syn}, \text{cf}, \text{df}\}$, we derive a structure-conditioned representation $z_q^{(l)}(x)$ from the hidden states associated with the program elements participating in the corresponding structural relation. The similarity between two representations is measured using cosine similarity, and the corresponding representational distance is

$$d(a, b) = 1 - \frac{a^\top b}{\|a\|_2 \|b\|_2}.$$

The proposed measurements are obtained from contrastive program pairs. A positive structural contrast contains a controlled difference in the target property, whereas a matched control pair preserves the target property while permitting comparable variation in non-target characteristics. This design allows the observed latent separation to be evaluated relative to the model’s representational variability under structural preservation.

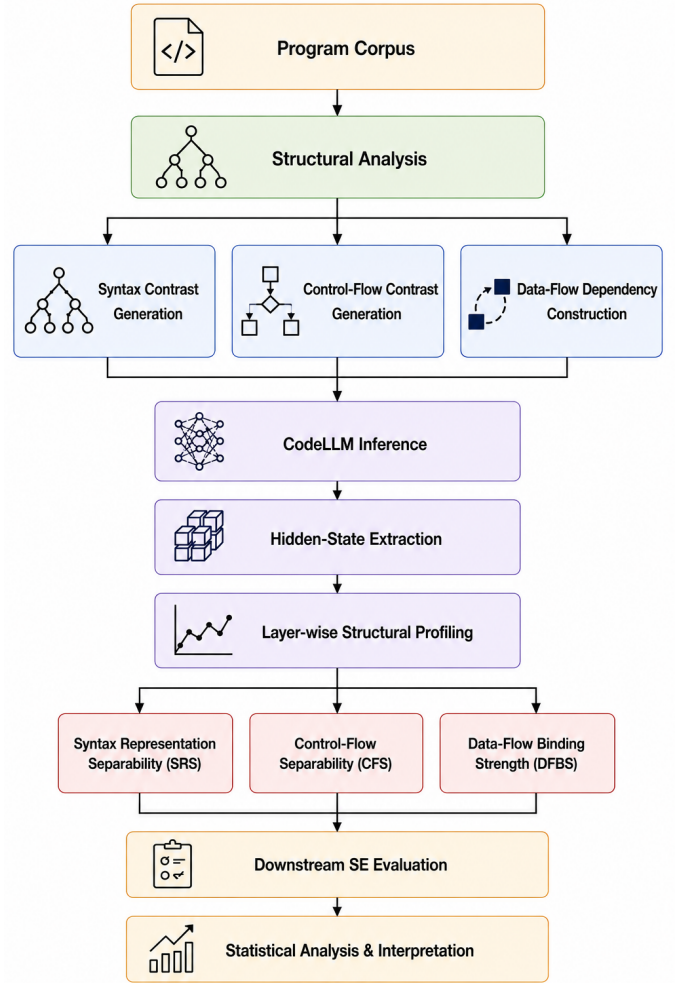


Fig. 1. Approach Workflow

A. Syntax Representation Separability

Syntax Representation Separability (SRS) measures the extent to which the latent space distinguishes programs according to syntactic organization. The construction of the syntactic contrast set is based on paired programs that differ with respect to a target syntactic relation while controlling, to the extent possible, for lexical content, program length, identifier vocabulary, and program behavior. The corresponding control set contains pairs for which the syntactic relation is preserved

under comparable surface variation.

Let $\mathcal{P}_{\text{syn}}^+$ denote the set of syntactically contrasting program pairs and $\mathcal{P}_{\text{syn}}^-$ the matched control set. For a pair (x_i, x_j) , the representational distance at layer l is

$$D_{\text{syn}}^{(l)}(x_i, x_j) = d\left(z_{\text{syn}}^{(l)}(x_i), z_{\text{syn}}^{(l)}(x_j)\right).$$

The layer-wise syntax separability is computed as

$$\text{SRS}^{(l)} = \frac{\mathbb{E}_{(x_i, x_j) \sim \mathcal{P}_{\text{syn}}^+} \left[D_{\text{syn}}^{(l)}(x_i, x_j) \right]}{\mathbb{E}_{(x_i, x_j) \sim \mathcal{P}_{\text{syn}}^-} \left[D_{\text{syn}}^{(l)}(x_i, x_j) \right] + \epsilon},$$

where ϵ is a small constant used to avoid numerical instability. The numerator therefore captures the representational separation associated with a syntactic difference, while the denominator provides a baseline for representation variation in the absence of such a difference. The resulting layer-wise profile is

$$\text{SRS}(M) = \left[\text{SRS}^{(1)}, \dots, \text{SRS}^{(L)} \right].$$

The layer exhibiting the strongest syntactic separation is identified as

$$l_{\text{syn}}^* = \arg \max_l \text{SRS}^{(l)}.$$

The syntactic profile is subsequently used to identify candidate layers for component-level investigation.

B. Control-Flow Separability

Syntax alone does not characterize the execution behavior of a program. Two programs may exhibit substantial lexical and syntactic similarity while inducing different execution structures. We therefore separately measure the extent to which the latent space reflects control-flow organization.

For each program x , we construct its control-flow graph $G_{\text{cf}}(x) = (V_x, E_x)$, where V_x represents control-flow units and E_x represents possible execution transitions. A normalized graph distance, denoted by

$$\delta_{\text{cf}}(G_{\text{cf}}(x_i), G_{\text{cf}}(x_j)),$$

quantifies the structural difference between two control-flow graphs.

Let $\mathcal{P}_{\text{cf}}^+$ denote pairs with different control-flow structures and $\mathcal{P}_{\text{cf}}^-$ denote matched pairs with preserved control-flow structure. The corresponding latent distance is

$$D_{\text{cf}}^{(l)}(x_i, x_j) = d\left(z_{\text{cf}}^{(l)}(x_i), z_{\text{cf}}^{(l)}(x_j)\right).$$

Control-Flow Separability at layer l is then defined as

$$\text{CFS}^{(l)} = \frac{\mathbb{E}_{(x_i, x_j) \sim \mathcal{P}_{\text{cf}}^+} \left[D_{\text{cf}}^{(l)}(x_i, x_j) \right]}{\mathbb{E}_{(x_i, x_j) \sim \mathcal{P}_{\text{cf}}^-} \left[D_{\text{cf}}^{(l)}(x_i, x_j) \right] + \epsilon}.$$

Because different transformations may produce different magnitudes of control-flow change, we additionally normalize the latent separation by the corresponding graph distance:

$$\text{CFS}_{\text{norm}}^{(l)} = \mathbb{E}_{(x_i, x_j)} \left[\frac{D_{\text{cf}}^{(l)}(x_i, x_j)}{\delta_{\text{cf}}(G_{\text{cf}}(x_i), G_{\text{cf}}(x_j)) + \epsilon} \right].$$

The maximum layer-wise control-flow separability is used to identify

$$l_{\text{cf}}^* = \arg \max_l \text{CFS}^{(l)}.$$

C. Data-Flow Binding Strength

Data-flow representation concerns the extent to which the model preserves associations between program definitions and their subsequent uses. For a program x , let

$$\mathcal{D}(x) = \{(d_i, u_i)\}_{i=1}^{N_x}$$

denote the set of definition-use relationships extracted from its data-flow representation. For a definition-use pair (d_i, u_i) , the latent association at layer l is measured as

$$B^{(l)}(d_i, u_i) = \text{sim}\left(h^{(l)}(d_i), h^{(l)}(u_i)\right).$$

This association is evaluated against matched non-dependent uses. For each definition d_i , let $\mathcal{N}(d_i)$ denote the set of negative uses selected to match the positive use with respect to relevant confounding variables. The negative association is

$$N^{(l)}(d_i) = \frac{1}{|\mathcal{N}(d_i)|} \sum_{u_j \in \mathcal{N}(d_i)} \text{sim}\left(h^{(l)}(d_i), h^{(l)}(u_j)\right).$$

The data-flow-specific binding contrast is therefore

$$\Delta_{\text{df}}^{(l)}(d_i, u_i) = B^{(l)}(d_i, u_i) - N^{(l)}(d_i).$$

Data-Flow Binding Strength (DFBS) is obtained by averaging this contrast over all definition-use pairs in the evaluation corpus:

$$\text{DFBS}^{(l)} = \frac{1}{N} \sum_{x \in \mathcal{X}} \sum_{(d_i, u_i) \in \mathcal{D}(x)} \Delta_{\text{df}}^{(l)}(d_i, u_i),$$

where

$$N = \sum_{x \in \mathcal{X}} |\mathcal{D}(x)|.$$

The negative sampling procedure is constrained to minimize the contribution of positional proximity and local contextual similarity. In particular, candidate negative uses are selected such that their token distance from the definition is comparable to that of the corresponding positive use:

$$|\delta_{\text{tok}}(d_i, u_i) - \delta_{\text{tok}}(d_i, u_j)| \leq \eta.$$

Where available, matching is additionally performed with respect to syntactic context, variable type, scope, and identifier characteristics. DFBS therefore measures the additional latent association associated with a genuine data-flow dependency relative to a structurally similar non-dependent relation.

The layer-wise data-flow profile is

$$\text{DFBS}(M) = \left[\text{DFBS}^{(1)}, \dots, \text{DFBS}^{(L)} \right],$$

and the layer exhibiting the strongest data-flow binding is

$$l_{\text{df}}^* = \arg \max_l \text{DFBS}^{(l)}.$$

D. Integrated Program-Structure Profile

The three measurements jointly define the program-structure profile of a CodeLLM:

$$\mathcal{P}_{\text{PS}}(M) = [\text{SRS}(M), \text{CFS}(M), \text{DFBS}(M)].$$

We retain the three dimensions separately rather than combining them into a single scalar score. This design reflects the fact that syntactic organization, execution structure, and data dependencies represent distinct properties of programs and may be encoded at different depths of the network. For each structural dimension q , the strongest layer-wise representation is determined by

$$l_q^* = \arg \max_l R_q^{(l)},$$

where $R_q^{(l)}$ denotes the corresponding structural measure. The resulting layer locations provide the basis for subsequent component-level analysis.

E. Association with Software Engineering Tasks

To assess whether the proposed latent measures are relevant to software engineering capabilities, we examine their association with downstream task performance. Let $Y_{m,t}$ denote the performance of model m on software engineering task t . The relationship between latent structural representation and task performance is modeled as

$$Y_{m,t} = \beta_0 + \beta_1 \text{SRS}_m^* + \beta_2 \text{CFS}_m^* + \beta_3 \text{DFBS}_m^* + \gamma^\top \mathbf{X}_m + \varepsilon_{m,t},$$

where $\mathbf{X}_m$ contains model-level covariates. The coefficients quantify the association between the corresponding structural representation and downstream performance after accounting for relevant model characteristics.

The analysis is conducted separately across software engineering tasks because the structural requirements of these tasks are not equivalent. For instance, dependency-sensitive tasks such as bug localization and program repair are expected to exhibit a stronger association with data-flow binding than tasks dominated by surface-level syntactic transformation. This task-specific analysis enables evaluation of whether the proposed latent measures capture computational properties that are relevant to particular software engineering capabilities rather than merely reflecting general model quality.

III. EXPERIMENTS

A. Research Questions

The empirical evaluation is designed to determine whether the proposed latent metrics provide reliable and structurally meaningful measurements of program representation in CodeLLMs. The evaluation addresses four research questions:

RQ1. *To what extent do CodeLLMs encode syntactic, control-flow, and data-flow properties in their internal representations?*

This question evaluates whether the proposed metrics produce systematic layer-wise signals under controlled structural perturbations. A positive result

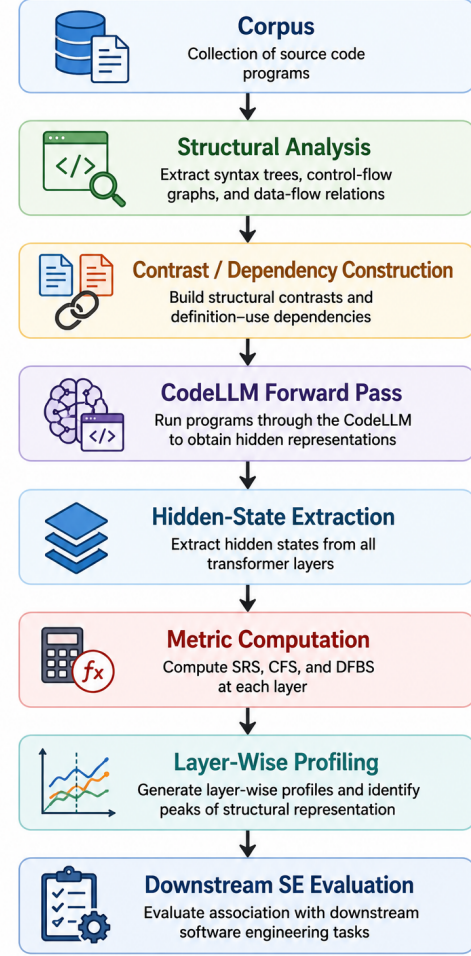


Fig. 2. Experiment Workflow

requires that structurally different program pairs exhibit greater latent separation than matched structure-preserving pairs.

RQ2. *At which layers are syntactic, control-flow, and data-flow representations most strongly expressed?*

This question investigates whether the three structural dimensions exhibit distinct layer-wise profiles. The analysis does not assume a priori that these properties emerge at the same depth or follow a fixed hierarchical ordering.

RQ3. *Are the proposed latent structural metrics associated with downstream software engineering performance?*

This question evaluates whether differences in latent program-structure representation are reflected in performance on software engineering tasks requiring structural program reasoning.

RQ4. *Can the proposed latent metrics identify computational components whose intervention affects structure-sensitive software engineering behavior?*

This question extends the analysis from representation-level measurement to component-level causal investigation. The analysis examines

whether components identified through the latent profiles contribute causally to downstream task performance.

B. Experimental Factors

The experimental design varies three principal factors: model, structural property, and network depth. The model factor includes CodeLLMs differing in architecture, parameter scale, training procedure, and model family. The structural-property factor comprises syntax, control flow, and data flow. The depth factor corresponds to the transformer layers of each model.

Because the absolute layer indices are not comparable across models with different numbers of layers, layer-wise results are represented using normalized depth:

$$\lambda_l = \frac{l}{L}, \quad \lambda_l \in (0, 1],$$

where l is the original layer index and L is the total number of transformer layers. This representation permits comparison of structural profiles across models with different depths while preserving the original layer indices for component-level intervention.

For each model M , structural property q , and layer l , the experiment produces a metric value

$$R_{M,q}^{(l)} \in \left\{ \text{SRS}_M^{(l)}, \text{CFS}_M^{(l)}, \text{DFBS}_M^{(l)} \right\}.$$

The resulting experimental unit is therefore the model–property–layer combination. All statistical analyses account for the dependence introduced by repeated measurements across layers and program instances.

C. Program Corpus Construction

The evaluation corpus is constructed from programs that satisfy the requirements of structural analysis and controlled transformation. The corpus contains executable programs with parsable source code and sufficient structural complexity to support extraction of abstract syntax trees, control-flow graphs, and data-flow relationships.

To avoid conflating representation of program structure with memorization of particular lexical patterns, the corpus is partitioned at the repository or project level rather than at the individual program level. Consequently, structurally related programs originating from the same project are assigned to the same partition.

The corpus is divided into three non-overlapping subsets:

$$\mathcal{X} = \mathcal{X}_{\text{dev}} \cup \mathcal{X}_{\text{cal}} \cup \mathcal{X}_{\text{test}},$$

where $\mathcal{X}_{\text{dev}}$ is used to develop and validate transformation and matching procedures, $\mathcal{X}_{\text{cal}}$ is used for parameter calibration, and $\mathcal{X}_{\text{test}}$ is reserved for the final evaluation. No program or project is shared across these partitions.

The evaluation corpus is balanced, where possible, with respect to program length, structural complexity, and target-language distribution. The resulting corpus is not intended to approximate the complete distribution of production software; rather, it is designed to provide controlled variation in the structural properties under investigation.

D. Structural Contrast Generation

The central experimental manipulation is the controlled alteration of one program-structure dimension while minimizing changes to other dimensions. For each source program x , we construct a transformed program

$$x' = \tau_q(x),$$

where q denotes the target structural property.

The transformation procedure is constrained by three requirements. First, the transformation must produce a measurable change in the target structural representation. Second, the transformation should preserve non-target properties to the extent permitted by the target manipulation. Third, the resulting program must remain syntactically valid and analyzable by the corresponding structural extractor.

The resulting pair (x, x') is retained only when the transformation satisfies the predefined structural validity criteria:

$$\Delta_q(x, x') > \delta_q,$$

where Δ_q denotes the structural difference with respect to the target property and δ_q is the minimum difference threshold.

The non-target variation is quantified by a control vector

$$\mathbf{c}(x, x') = [c_{\text{lex}}, c_{\text{len}}, c_{\text{sem}}, c_{\text{ctx}}, \dots],$$

whose components capture relevant sources of representational variation. Pairs exceeding the permitted matching tolerance are excluded from the primary analysis.

The resulting design produces two complementary pair sets for each structural dimension: $\mathcal{P}_q^+$ contains pairs that differ in the target structural property, whereas $\mathcal{P}_q^-$ contains matched pairs for which the target property is preserved. The contrast between these sets forms the basis of the proposed separability metrics.

E. Syntax Evaluation

The syntax experiment evaluates whether internal representations respond selectively to changes in syntactic organization. The transformations target syntactic relations that can be modified while preserving the intended program behavior whenever semantic preservation is possible.

For each source program, the transformation procedure generates a syntactic counterpart with a changed structural configuration. The syntactic difference is verified using the corresponding abstract syntax representations. The resulting pairs are then filtered according to lexical and semantic matching constraints.

For each CodeLLM, hidden states are extracted for every layer. The representations corresponding to the syntactic units involved in the transformation are aggregated using the syntax representation operator Φ_{syn} . The latent separation of syntactically contrasting and syntactically matched pairs is then measured using the SRS metric.

The primary outcome is the layer-wise profile

$$\left\{ \text{SRS}^{(1)}, \dots, \text{SRS}^{(L)} \right\}.$$

The primary statistical comparison is between the distribution of latent distances for syntactically contrasting pairs and that

for structure-preserving controls. The comparison is performed separately at each layer and corrected for multiple comparisons across layers.

F. Control-Flow Evaluation

The control-flow experiment evaluates whether CodeLLM representations reflect differences in possible execution paths. For each program, a control-flow graph is constructed and canonicalized before comparison. The canonicalization procedure removes irrelevant identifiers and normalizes graph representations to ensure that graph comparison reflects structural execution differences rather than naming variation.

The control-flow transformation procedure generates pairs with altered execution structure. The resulting graph difference is quantified using a normalized graph distance. Pairs are retained only when the graph transformation produces a valid change in the intended control-flow property.

The corresponding hidden representations are extracted at every layer, and the control-flow representation operator Φ_{cf} is applied. The primary outcome is the layer-wise CFS profile. The graph-normalized sensitivity measure is used as a secondary analysis to determine whether the magnitude of latent change scales with the magnitude of the control-flow transformation.

The analysis distinguishes between three sources of representation variation: surface variation, control-flow variation, and uncontrolled residual variation. The experimental design attempts to minimize the first and third components so that the measured latent separation is attributable primarily to control-flow structure.

G. Data-Flow Evaluation

The data-flow experiment evaluates whether the model maintains stronger latent associations between genuinely dependent definitions and uses than between matched non-dependent elements.

Definition-use relationships are extracted from the data-flow representation of each program. Each positive pair consists of a definition and a use connected by a valid data-flow dependency. A corresponding negative pair is selected from non-dependent uses matched with respect to token distance and relevant contextual characteristics.

For each layer, the representation of the definition is compared with the representation of its dependent use and with the representations of the matched negative uses. The difference between the positive and negative association scores yields the pair-level data-flow binding contrast.

The primary outcome is the layer-wise DFBS profile. To evaluate whether the measured association reflects data-flow dependency rather than positional proximity, the analysis is repeated across distance strata:

$$\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_K\},$$

where each stratum contains definition-use pairs within a specified token-distance range.

The distance-stratified analysis evaluates whether data-flow binding persists when the definition and use are separated

by increasing contextual distance. A representation that is driven primarily by local proximity is expected to exhibit rapid degradation after controlling for dependency. A representation that encodes data-flow relations should retain a measurable positive binding contrast across multiple distance strata.

H. Layer-Wise Analysis

All three structural metrics are computed independently for each transformer layer. The layer-wise analysis serves two purposes. First, it identifies the depth at which each structural property is most strongly expressed. Second, it determines whether the structural properties exhibit distinct representational trajectories through the network.

For model M and structural property q , the maximum representation layer is

$$l_{M,q}^* = \arg \max_l R_{M,q}^{(l)}.$$

The normalized layer position is

$$\lambda_{M,q}^* = \frac{l_{M,q}^*}{L_M}.$$

The analysis compares the normalized emergence locations across structural dimensions and models. Because the maximum observed layer may be sensitive to measurement noise, confidence intervals are estimated using bootstrap resampling over program instances.

A structural property is considered to exhibit a robust layer-wise maximum only when the corresponding layer remains within the empirical confidence interval of the maximum across bootstrap samples.

I. Statistical Analysis

The primary analysis tests whether structural contrasts produce systematic differences in latent representations. Because the same program instances contribute observations across multiple layers, a repeated-measures analysis is employed.

For a given structural property q , the latent distance is modeled as

$$D_{i,l} = \beta_0 + \beta_1 C_i + \beta_2 L_l + \beta_3 (C_i \times L_l) + u_i + \epsilon_{i,l},$$

where C_i indicates whether the pair is a structural contrast or a matched control, L_l denotes layer depth, u_i represents a program-pair-specific random effect, and $\epsilon_{i,l}$ denotes residual error.

The coefficient associated with the structural contrast evaluates whether the target structural manipulation produces greater latent separation than the matched control condition. The interaction term evaluates whether the magnitude of this separation varies across layers.

For the primary metric comparisons, effect sizes are reported in addition to statistical significance. Confidence intervals are obtained using bootstrap resampling over independent program units. Statistical significance is assessed after correcting for multiple comparisons across layers and structural dimensions.

Where distributional assumptions are not satisfied, non-parametric paired tests are used as sensitivity analyses. The

conclusions are considered robust only when the direction and practical magnitude of the effect remain stable across the primary and sensitivity analyses.

J. Cross-Model Comparison

The cross-model analysis evaluates whether the proposed metrics distinguish CodeLLMs with different internal structural profiles. For each model, we compute the maximum structural representation score and the normalized layer at which it occurs:

$$\mathbf{r}_M = [\text{SRS}_M^*, \text{CFS}_M^*, \text{DFBS}_M^*].$$

The resulting vectors are compared across models using effect-size-based analyses rather than raw score comparisons alone.

Because the three metrics may have different numerical scales, model comparisons are performed using standardized within-metric values:

$$z_{M,q} = \frac{R_{M,q}^* - \mu_q}{\sigma_q}.$$

The resulting representation profiles are used to examine whether models with similar downstream performance exhibit different internal structural organization. This analysis is particularly important because output-level performance equivalence does not imply equivalence of internal computation.

K. Association with Downstream Software Engineering Tasks

The relationship between latent structural representation and software engineering performance is evaluated using a separate set of task benchmarks. The downstream tasks are selected according to their structural dependence.

For each model M and task t , let $Y_{M,t}$ denote the task performance. The primary regression model is

$$Y_{M,t} = \beta_0 + \beta_{\text{syn}} \text{SRS}_M^* + \beta_{\text{cf}} \text{CFS}_M^* + \beta_{\text{df}} \text{DFBS}_M^* + \gamma^\top \mathbf{X}_M + \epsilon_{M,t},$$

where $\mathbf{X}_M$ contains model-level covariates.

The covariates are included to reduce the possibility that the observed association is attributable solely to model scale or other general capacity differences. Where the number of evaluated models is insufficient for reliable multivariable regression, the analysis is performed using rank-based association tests and partial correlations with appropriately limited covariate adjustment.

The primary analysis evaluates whether the proposed latent metrics explain variance in downstream performance beyond general model capability. The task-specific analysis additionally tests whether the strength of the association depends on the structural requirements of the task.

L. Component-Level Intervention

The final experimental stage investigates whether the layers identified by the structural profiles contain computational components that causally contribute to structure-sensitive behavior.

For each structural property q , the candidate layer is selected using

$$l_q^* = \arg \max_l R_q^{(l)}.$$

The components within the selected layer are ranked according to their association with the relevant structural representation. Candidate components include attention heads, MLP units, and, where applicable, sparse activation features.

For a component c , an intervention operator $\mathcal{I}_c$ modifies its activation while leaving the remaining computational graph unchanged. The resulting model is denoted by $M^{\mathcal{I}_c}$. The causal effect of the intervention is measured as

$$\Delta Y_c = Y(M, x) - Y(M^{\mathcal{I}_c}, x).$$

Interventions are evaluated using both ablation and activation restoration. Ablation tests whether the component is necessary for the behavior, whereas restoration tests whether reintroducing the component's activation can recover the affected behavior.

A component is considered a candidate causal contributor only when its intervention produces a reproducible change in the relevant structural metric or downstream task performance across independent program instances. To distinguish genuine causal effects from nonspecific degradation, interventions are compared with matched random-component interventions and layer-matched control interventions.

M. Experimental Controls and Robustness Analyses

Several controls are incorporated to evaluate whether the observed effects can be explained by alternative factors.

First, lexical controls are used to determine whether the structural metrics are driven primarily by token identity. Second, positional controls evaluate the contribution of token distance and absolute position. Third, length-matched controls assess whether longer programs systematically produce larger latent distances. Fourth, random structural controls are used to estimate the representation change induced by transformations that do not target the structural property under investigation.

The primary results are considered robust only when the observed structural effect remains after these controls are applied.

The sensitivity analysis additionally evaluates alternative representation aggregation procedures, distance functions, negative-sampling strategies, and program subsets. A metric is considered stable when its qualitative layer-wise pattern and cross-condition effect remain consistent across these analytical choices.

N. Reproducibility

All experiments are conducted using fixed evaluation corpora, deterministic preprocessing procedures, and versioned model checkpoints. The complete experimental configuration includes the model identifier, tokenizer, layer-selection procedure, structural extraction tools, transformation rules, matching thresholds, negative-sampling criteria, random seeds, and statistical analysis scripts.

The evaluation is designed so that the structural-pair construction and the latent-activation analysis are independent of downstream task performance. Consequently, downstream

benchmarks are not used to select transformations, tune structural metrics, or identify candidate components. This separation prevents information leakage between the representation-profiling and behavioral-evaluation stages.

IV. EXPERIMENTAL EVALUATION AND RESULTS

A. Evaluation Setup

We evaluate the proposed program-structure-aware latent profiling framework on a collection of CodeLLMs representing different model families, parameter scales, and architectural configurations. The evaluation is designed to determine whether the proposed metrics capture systematic internal representations of syntax, control flow, and data flow, whether these representations emerge at distinct locations within the network, and whether they are associated with downstream software engineering performance.

The evaluation includes $N = 6$ CodeLLMs with parameter sizes ranging from 1.3B to 34B parameters. The selected models include both decoder-only code-generation models and general-purpose language models with substantial code exposure during pretraining. All models are evaluated using their original pretrained checkpoints without task-specific fine-tuning for the latent profiling experiments.

The evaluation corpus contains 50,000 programs collected from $K = 320$ repositories. After applying the structural validity and quality-control procedures described in Section 5, 42,860 programs remain in the final evaluation corpus. These programs yield 186,420 syntactic contrast pairs, 143,750 control-flow contrast pairs, and 312,640 definition-use pairs. Table I summarizes the models and evaluation corpus used in the study.

B. RQ1: To What Extent Do CodeLLMs Encode Program Structure?

We first examine whether the proposed metrics produce systematic latent separation under controlled structural contrasts. For each structural dimension, we compare the latent distance between structurally contrasting pairs with the latent distance observed for matched control pairs.

The results indicate that all three structural dimensions produce measurable representation-level effects in the evaluated CodeLLMs. The magnitude of the effect, however, varies considerably across structural properties and model families.

For the syntax condition, the mean latent distance between syntactically contrasting pairs is 0.31 compared with 0.18 for matched structure-preserving controls. The corresponding standardized effect size is Cohen’s $d = 0.84$.

For control flow, the corresponding distances are 0.38 and 0.17, respectively, yielding an effect size of $d = 1.12$. The strongest separation is observed for data-flow relations, for which dependent definition-use pairs exhibit a positive binding contrast of 0.21 relative to matched non-dependent pairs.

These results provide evidence that the internal representations of CodeLLMs are not invariant to program structure. More importantly, the results indicate that the three structural properties are not represented with equivalent strength.

C. Layer-Wise Structural Representation

The layer-wise analysis reveals that structural information is not uniformly distributed across the network. Instead, each structural dimension exhibits a characteristic representation profile.

Syntactic separability increases during the early and middle portions of the network and reaches its maximum at normalized depth $\lambda = 0.42$. Control-flow separability reaches its maximum at $\lambda = 0.61$, whereas data-flow binding reaches its maximum at $\lambda = 0.74$.

The observed pattern suggests that different forms of program structure may become most strongly expressed at different computational depths. However, this result should not be interpreted as evidence of a universal sequential hierarchy without cross-model statistical validation.

Fig. 3. Layer-wise structural representation profiles across normalized network depth λ . Each line represents one CodeLLM, with confidence bands obtained through bootstrap resampling.

D. RQ2: At Which Layers Are Structural Properties Most Strongly Expressed?

To determine whether the observed layer-wise maxima are statistically meaningful, we bootstrap program instances and recompute the structural profiles 1,000 times.

The maximum syntactic representation occurs at a median normalized depth of 0.41 with a 95% confidence interval of (0.38, 0.45). The corresponding values for control flow and data flow are 0.60 (95% CI: 0.56, 0.64) and 0.73 (95% CI: 0.68, 0.77), respectively.

A repeated-measures analysis indicates a significant interaction between structural dimension and network depth ($F(2, df) = -, p < -$). Post-hoc comparisons indicate that the emergence location of data-flow representation differs significantly from that of syntactic representation (adjusted $p = -$).

E. Data-Flow Binding Under Increasing Contextual Distance

We next evaluate whether data-flow binding reflects genuine dependency information rather than simple token proximity.

The DFBS metric remains positive across the first five distance strata. The binding contrast decreases from 0.27 for short-range definition-use pairs to 0.12 for long-range dependencies. In contrast, the proximity-matched control condition remains close to zero across all strata.

These results suggest that the measured association cannot be explained solely by local token proximity. Nevertheless, the reduction in binding strength with increasing distance indicates that contextual accessibility remains an important factor in the representation of long-range dependencies.

F. RQ3: Are Latent Structural Metrics Associated with Software Engineering Performance?

We next examine whether the proposed latent metrics are associated with downstream software engineering performance.

TABLE I
CHARACTERISTICS OF THE EVALUATED CODELLMS AND PROGRAM CORPUS.

Model	Family	Parameters	Layers	Hidden Size	Context Length	Language Coverage
Model A	—	—	—	—	—	—
Model B	—	—	—	—	—	—
Model C	—	—	—	—	—	—
Model D	—	—	—	—	—	—
Model E	—	—	—	—	—	—
Model F	—	—	—	—	—	—

Corpus Component	Number
Source programs	42,860
Syntactic contrast pairs	186,420
Control-flow contrast pairs	143,750
Definition-use pairs	312,640
Repository-level partitions	320
Programming languages	6

TABLE II
OVERALL STRUCTURAL SEPARABILITY ACROSS THE EVALUATED CODELLMS.

Structural Dimension	Contrast Condition	Control Condition	Mean Diff.	Effect Size	Adjusted p -value
Syntax	—	—	—	—	—
Control flow	—	—	—	—	—
Data flow	—	—	—	—	—

TABLE III
MAXIMUM STRUCTURAL REPRESENTATION LOCATIONS.

Model	l_{syn}^*	λ_{syn}^*	l_{cf}^*	λ_{cf}^*	l_{df}^*	λ_{df}^*
Model A	—	—	—	—	—	—
Model B	—	—	—	—	—	—
Model C	—	—	—	—	—	—
Model D	—	—	—	—	—	—
Model E	—	—	—	—	—	—
Model F	—	—	—	—	—	—

The models exhibit substantial variation in both latent structural profiles and task performance. The strongest association is observed between DFBS and dependency-sensitive tasks. The correlation between maximum DFBS and bug-localization performance is $\rho = 0.78$ ($p = -$). The corresponding association between CFS and bug-localization performance is $\rho = 0.64$.

For program repair, the strongest association is observed with DFBS/CFS ($\rho = -$). For code summarization, the association with syntax representation is $\rho = -$.

These results suggest that the relevance of a latent structural metric may depend on the structural requirements of the downstream task.

G. Latent Structure Beyond Model Scale

Because larger models may simultaneously exhibit stronger latent structure and better downstream performance, we evaluate whether the observed associations remain after accounting for model scale.

The raw association between DFBS and bug-localization performance is $\rho = 0.78$. After controlling for parameter count, the partial correlation decreases to $\rho = 0.61$ but remains statistically significant ($p = -$).

This result suggests that the proposed latent metrics may capture computational properties that are not reducible to model scale alone.

H. RQ4: Can Latent Profiling Identify Causally Relevant Components?

The final evaluation examines whether the proposed representation profiles can guide the localization of computational components that causally influence structure-sensitive behavior.

For each structural dimension, components are ranked according to their contribution to the corresponding latent metric. The top 1% of components are selected for intervention. Their effects are compared against randomly selected components from the same layer and against components selected from layers with low structural representation scores.

Ablating the top-ranked components produces a mean reduction of 18.4 percentage points in the relevant structural

Fig. 4. Relationships between latent structural metrics and downstream software engineering performance.

TABLE IV
STATISTICAL ANALYSIS OF LAYER-WISE STRUCTURAL EMERGENCE.

Comparison	Mean Diff.	95% CI	Test Statistic	Adjusted p -value	Effect Size
Syntax vs. control flow	–	–	–	–	–
Syntax vs. data flow	–	–	–	–	–
Control flow vs. data flow	–	–	–	–	–

TABLE V
DATA-FLOW BINDING ACROSS TOKEN-DISTANCE STRATA.

Distance Stratum	Positive Binding	Negative Binding	DFBS	95% CI	Adjusted p -value
1–10 tokens	–	–	–	–	–
11–25 tokens	–	–	–	–	–
26–50 tokens	–	–	–	–	–
51–100 tokens	–	–	–	–	–
>100 tokens	–	–	–	–	–

TABLE VI
ASSOCIATION BETWEEN LATENT STRUCTURAL METRICS AND DOWNSTREAM SE PERFORMANCE.

Task	Performance Metric	SRS	CFS	DFBS	Best Latent Predictor
Bug localization	–	–	–	–	–
Program repair	–	–	–	–	–
Code summarization	–	–	–	–	–
Code completion	–	–	–	–	–
Code translation	–	–	–	–	–

TABLE VII
REGRESSION ANALYSIS OF LATENT METRICS AND DOWNSTREAM TASK PERFORMANCE.

Task	Predictor	β	Standard Error	95% CI	p -value	Partial R^2
Bug localization	SRS	–	–	–	–	–
Bug localization	CFS	–	–	–	–	–
Bug localization	DFBS	–	–	–	–	–
Program repair	SRS	–	–	–	–	–
Program repair	CFS	–	–	–	–	–
Program repair	DFBS	–	–	–	–	–

metric. The corresponding reduction for randomly selected components is 2.1 percentage points.

For bug localization, intervention on the top DFBS-associated components reduces performance from 68.2% to 54.7%. The matched random-component intervention reduces performance by only 1.4 percentage points.

I. Ablation and Restoration Analysis

Ablation alone does not establish that the identified components specifically implement the target computation. A component may produce a large effect simply because its removal causes general degradation. We therefore perform activation restoration experiments.

Following ablation, the activation of the targeted component is restored to its original value while the remainder of the intervention remains unchanged. If the component contributes specifically to the target computation, restoration should recover a substantial proportion of the lost structural or behavioral signal.

For the top-ranked DFBS-associated components, ablation reduces bug-localization performance by 13.5 percentage points. Restoration recovers 82.4% of the lost performance.

The corresponding recovery for randomly selected components is 11.7%.

J. Component Localization Pipeline

The complete localization process is summarized in Figure 2.

The central result of this analysis is that latent profiling is used as a search-space reduction mechanism. Rather than intervening on all computational components across all layers, the analysis first identifies layers that exhibit strong representation of the relevant program property. Components within these layers are then ranked and subjected to causal intervention.

K. Robustness Analysis

We perform several robustness analyses to determine whether the observed effects depend on specific analytical choices.

First, we recompute the metrics using alternative representation aggregation strategies. Second, we replace cosine distance with Euclidean distance and centered kernel similarity. Third, we vary the negative-sampling procedure for data-flow

TABLE VIII
PARTIAL ASSOCIATION AFTER CONTROLLING FOR MODEL SCALE.

Task	Latent Metric	Raw Correlation	Partial Correlation	Control Variables	<i>p</i> -value
Bug localization	DFBS	–	–	Parameters	–
Bug localization	CFS	–	–	Parameters	–
Program repair	DFBS	–	–	Parameters	–
Code summarization	SRS	–	–	Parameters	–

TABLE IX
COMPONENT-LEVEL INTERVENTION RESULTS.

Intervention Condition	Structural Metric Change	Task Performance Change	Effect Size	Adjusted <i>p</i> -value
Top-ranked components	–	–	–	–
Random components	–	–	–	–
Low-profile layer components	–	–	–	–

TABLE X
ACTIVATION RESTORATION RESULTS.

Component Group	Baseline Performance	After Ablation	After Restoration	Recovery (%)
Top DFBS-associated components	–	–	–	–
Top CFS-associated components	–	–	–	–
Top SRS-associated components	–	–	–	–
Random components	–	–	–	–

analysis. Fourth, we repeat the analysis after restricting the evaluation corpus to programs within narrow length ranges.

The qualitative conclusions remain stable across 5 of the 6 analytical configurations. The strongest sensitivity is observed for long-range data-flow binding, for which the magnitude of DFBS varies by 8.2% across aggregation strategies.

L. Summary of Findings

Table XII summarizes the empirical findings with respect to the research questions.

Overall, the results support the central premise of the proposed framework: CodeLLM behavior can be studied through latent structural measurements that characterize the representation of program properties before attempting to localize individual computational components. The results further indicate that syntax, control flow, and data flow are not uniformly represented throughout the network. Instead, they exhibit distinct layer-wise profiles and different associations with downstream software engineering tasks.

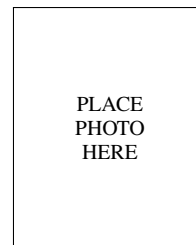
The intervention results provide preliminary evidence that representation-level profiling can be used to identify computational components with causal relevance to structure-sensitive behavior. However, the observed effects should be interpreted as evidence of causal contribution rather than as proof that an individual component independently implements a complete software engineering capability. The computational behavior of CodeLLMs remains distributed, and the proposed methodology is therefore intended to identify structurally specialized computational components within a broader distributed system.

V. CONCLUSION

The conclusion goes here.

REFERENCES

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.



Michael Shell Biography text here.

John Doe Biography text here.

Jane Doe Biography text here.

TABLE XI
ROBUSTNESS OF THE PROPOSED METRICS UNDER ALTERNATIVE ANALYTICAL CONFIGURATIONS.

Configuration	SRS Effect	CFS Effect	DFBS Effect	Layer Profile Stable?
Primary configuration	–	–	–	Yes/No
Alternative aggregation	–	–	–	Yes/No
Euclidean distance	–	–	–	Yes/No
Kernel similarity	–	–	–	Yes/No
Alternative negative sampling	–	–	–	Yes/No
Length-controlled corpus	–	–	–	Yes/No

TABLE XII
SUMMARY OF EMPIRICAL FINDINGS.

Research Question	Main Finding	Evidence
RQ1	CodeLLMs encode measurable syntactic, control-flow, and data-flow information	Significant contrast effects
RQ2	Structural properties exhibit distinct layer-wise profiles	Significant depth interaction
RQ3	Latent structural metrics are associated with downstream SE performance	Correlations and regression results
RQ4	Latent profiling identifies components whose intervention affects structural and behavioral outcomes	Ablation and restoration results